

Getting Started with C++

Content notes, classwork and homework with worked C++ solutions.

Lesson 1: Structure of a C++ Program

Quick recap: Structure of a C++ program • #include, main() • cout, endl, comments • Compile & run with g++

Theory & Nuances

Structure of a C++ program

Every C++ program needs exactly one function called `main()` — this is the entry point. No matter how many other functions exist in the file, execution always starts at the first line inside `main()` and ends when `main()` returns. The bare minimum skeleton is: an `#include`, then `int main() { ... return 0; }`.

#include and iostream

`#include` is a preprocessor directive — before your code is even compiled, the preprocessor pastes in the contents of the named file. `#include` pulls in the declarations for `cin` and `cout`, which is why leaving it out makes `cout` "undeclared". `main()` must return an `int`; returning 0 is the convention for "the program finished successfully".

cout, endl, and comments

`cout` is the standard output stream, and `<<` is the insertion operator — it pushes whatever is on its right into the stream, and you can chain several `<<` in one statement. `endl` moves to a new line AND flushes the output buffer; `"\n"` also moves to a new line but is slightly cheaper since it skips the flush. `//` starts a line comment, and `/* ... */` wraps a block comment — both are ignored by the compiler and exist purely to explain code to a human reader.

Compiling and running with g++

`g++ file.cpp -o file` turns your source code into an executable binary; `./file` then runs it. These are two separate steps — compilation only checks that the code is valid C++ (syntax errors show up here), while logic errors (wrong output) only show up once you actually run the program.

Key Points / Common Mistakes

- Forgetting `using namespace std;` means every `cout/cin` needs the `std::` prefix (`std::cout`).
- Every statement needs a semicolon — a missing one is the most common first compiler error.
- Mismatched `{ }` braces will cause confusing errors far below the actual mistake.
- `endl` flushes the buffer (slightly slower); `"\n"` doesn't — prefer `"\n"` in tight loops.

Classwork Problems

1. Print "Hello, World!"

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, World!" << endl;
    return 0;
}
```

2. Print your name, class and roll on separate lines

```
#include <iostream>
```

```
using namespace std;

int main() {
    cout << "Name : Rahul Sharma" << endl;
    cout << "Class : 9th - A" << endl;
    cout << "Roll : 21" << endl;
    return 0;
}
```

3. Print a small ASCII box

```
#include <iostream>
using namespace std;

int main() {
    cout << "+-----+" << endl;
    cout << "|          |" << endl;
    cout << "|          |" << endl;
    cout << "+-----+" << endl;
    return 0;
}
```

Homework Problems

1. Print your daily timetable as text

```
#include <iostream>
using namespace std;

int main() {
    cout << "----- My Daily Timetable -----" << endl;
    cout << "9:00 - 9:45 : Mathematics" << endl;
    cout << "9:45 - 10:30 : Science" << endl;
    cout << "10:30 - 10:45 : Break" << endl;
    cout << "10:45 - 11:30 : English" << endl;
    cout << "11:30 - 12:15 : Computer Science" << endl;
    return 0;
}
```

2. Print a 4-line ASCII figure

```
#include <iostream>
using namespace std;

int main() {
    cout << "  /\\" << endl;
    cout << " /  \\" << endl;
    cout << " /____\\" << endl;
    cout << " |    | " << endl;
    return 0;
}
```

Lesson 2: Data Types & Variables

Quick recap: int, long long, double, char, bool, string • Declaration vs initialization • sizeof and value ranges • const and auto

Theory & Nuances

The core data types

int stores whole numbers (roughly ± 2 billion); long long is used when a number can exceed that range (populations, factorials, big sums). double stores decimal/floating-point values. char stores a single character, but is really stored as a small integer (its ASCII code). bool stores only true or false. string stores text — it isn't a built-in primitive like the others, it's a class, so it needs #include (or comes bundled via in most setups).

Declaration vs initialization

Declaration just reserves memory for a variable: `int x;` — at this point `x` holds an unpredictable garbage value, not 0. Initialization gives it a starting value at the same time: `int x = 5;`. Always initialize variables — reading an uninitialized variable is a classic source of bugs that behave differently each run.

sizeof and value ranges

Every type uses a fixed number of bytes in memory — `sizeof(type)` reports it. Because the size is fixed, every type also has a limited range of values it can hold. Going past that range causes overflow, which does NOT crash the program or throw an error — it silently wraps around to a wrong (often negative) number, making it a dangerous, quiet bug.

const and auto

`const` locks a variable after initialization — any later attempt to reassign it is a compile error. It's used for values that should never change, like a PI constant. `auto` tells the compiler "figure out the type yourself from the right-hand side" — handy for long/complex types, but should still be used where the type is obvious from context so the code stays readable.

Key Points / Common Mistakes

- An uninitialized variable holds garbage, never a guaranteed 0.
- Integer overflow wraps silently instead of erroring — always sanity-check large calculations.
- Double-quoted "text" is a string; single-quoted 'a' is a char — mixing these up is a common typo.
- `const` variables must be given a value at the moment they're declared.

Classwork Problems

1. Declare and print a variable of each type

```
#include <iostream>
using namespace std;

int main() {
    int age = 15;
    long long population = 1400000000LL;
    double pi = 3.14159;
    char grade = 'A';
    bool isPassed = true;
    string name = "Aditi";

    cout << "int      : " << age << endl;
    cout << "long long: " << population << endl;
    cout << "double   : " << pi << endl;
    cout << "char     : " << grade << endl;
    cout << "bool    : " << isPassed << endl;
    cout << "string  : " << name << endl;
    return 0;
}
```

2. Area and perimeter of a rectangle

```
#include <iostream>
using namespace std;

int main() {
    double length = 12.5, breadth = 8.0;

    double area = length * breadth;
    double perimeter = 2 * (length + breadth);

    cout << "Area      : " << area << endl;
    cout << "Perimeter : " << perimeter << endl;
    return 0;
}
```

3. Swap two numbers using a third variable

```
#include <iostream>
using namespace std;

int main() {
    int a = 5, b = 9, temp;

    cout << "Before swap: a = " << a << ", b = " << b << endl;

    temp = a;
    a = b;
    b = temp;

    cout << "After swap : a = " << a << ", b = " << b << endl;
    return 0;
}
```

Homework Problems

1. Print sizeof of each type

```
#include <iostream>
using namespace std;

int main() {
    cout << "sizeof(int) : " << sizeof(int) << " bytes" << endl;
    cout << "sizeof(long long) : " << sizeof(long long) << " bytes" << endl;
    cout << "sizeof(double) : " << sizeof(double) << " bytes" << endl;
    cout << "sizeof(char) : " << sizeof(char) << " bytes" << endl;
    cout << "sizeof(bool) : " << sizeof(bool) << " bytes" << endl;
    cout << "sizeof(string) : " << sizeof(string) << " bytes" << endl;
    return 0;
}
```

2. Celsius to Fahrenheit converter

```
#include <iostream>
using namespace std;

int main() {
    double celsius = 37.0;
    double fahrenheit = (celsius * 9.0 / 5.0) + 32.0;

    cout << celsius << " C = " << fahrenheit << " F" << endl;
    return 0;
}
```
